

Team Retrospective

news for u ^_^

SE 4450 — Software Engineering Design II

Agile Alligators

Vivek Jariwala | Kaura Sankaran | Sharaf Syed | Scarlet Wu

Faculty Advisor: Dr. Yimin Yang | **Co-Advisor:** Dr. Elvis Chen

April 6, 2026

1. Process Overview

The Agile Alligators are a four-person team that followed the Scrum framework across two sprints to build “news for u,” an AI-powered news platform for the SE 4450 capstone course at Western University. Vivek Jariwala served as Product Owner, responsible for backlog ordering, acceptance criteria, and integrating advisor feedback. Kaura Sankaran served as Scrum Master, facilitating ceremonies, removing impediments, and tracking improvement actions. All four members—Vivek, Kaura, Sharaf Syed, and Scarlet Wu—acted as cross-functional developers contributing to frontend, backend, AI integration, and QA.

Our process included stand-ups three times per week (Tuesdays and Thursdays at 5 PM EST, Saturdays at noon), biweekly sprint planning, sprint reviews with our advisors, and retrospectives at the end of each sprint. Between sprints, we held break meetings to align on research and carry-over items.

Our tooling consisted of GitHub for version control and code review (with protected branches and pull-request reviews), Jira for task tracking and sprint boards, Discord for day-to-day async communication, and Microsoft Teams for document collaboration. We used Postman for backend API testing and manual UI walkthroughs for system-level validation.

A notable process challenge was that two team members (Kaura and Scarlet) were on international exchange during Sprint 2, introducing three different time zones. This forced us to move from loosely scheduled meetings to a fixed weekly cadence with stronger individual ownership.

2. Sprint / Iteration Review

Sprint 1 (September–December 2025)

Sprint 1 focused on building the end-to-end platform foundation: user authentication, the news ingestion pipeline (via CurrentsAPI), article categorization and timestamping, database and infrastructure setup, and the complete React UI shell. We deliberately prioritized the data flow from external APIs through the backend to the frontend before introducing AI features, reducing integration risk later.

All planned high-priority PBIs (Stories 1.1, 1.2, 2.1–2.4, 4.1, 5.2, 5.3) were completed. Several AI-related stories (3.1–3.4) were started at the UI level to support early usability testing, but full backend integration was intentionally deferred. LLM integration was the primary carry-over, as the team had not yet settled on a cost-effective approach.

Sprint 2 (January–March 2026)

Sprint 2 layered intelligence and personalization onto the stable foundation. Major deliverables included: Google Gemini-powered article summaries and explanations, Hugging Face readability/complexity scoring, embedding generation (via gemini-embedding-001 with pgvector), learning roadmap generation, a switch to the Guardian News API, personalized headline feeds, the clippings system with shareable public links, and full admin user management.

Epic 6 (Feedback & Learning Loop) was partially prototyped but not fully shipped due to time constraints. Epic 7 (Notifications & Alerts) was formally dropped from scope after reassessment—the team determined it introduced complexity without contributing directly to the platform’s core AI-driven value proposition.

Summary Table

Sprint	Planned Deliverables	Actual Deliverables	Key Issues / Variances
Sprint 1	User auth, news pipeline, categorization, timestamps, full UI shell, initial AI exploration	All core PBIs delivered. UI prototypes for AI features built. No LLM integration yet.	LLM approach undefined. CurrentsAPI lacked article body content. Meeting scheduling inconsistent.

Sprint 2	AI integration (Gemini + HF), recommendations, feedback loops, alerts, admin features, clippings	AI summaries, complexity scoring, embeddings, roadmaps, Guardian API, clippings, admin panel delivered. Feedback loop partially prototyped.	Notifications dropped. Feedback loop incomplete. Time zone challenges with 2 members on exchange. Per-topic expertise levels deferred.
----------	--	---	--

3. What Worked Well

3.1 Strong Foundation-First Architecture Strategy

The decision to build the complete end-to-end data pipeline and UI shell in Sprint 1—before introducing any AI features—paid significant dividends. When Sprint 2 began and we integrated Google Gemini and Hugging Face models, the infrastructure was already stable: APIs were tested, the database schema was in place, and the frontend routing was complete. This meant AI integration could proceed without rework of foundational components. For example, adding embedding storage required only a pgvector extension to our existing PostgreSQL setup, not a schema redesign.

3.2 Effective Individual Ownership Under Distributed Conditions

With two members on exchange in different time zones during Sprint 2, the team adapted by assigning clear individual ownership of features. Each person identified their responsibilities and delivered independently, with ad-hoc coordination calls when blockers arose. This autonomy kept the sprint moving without requiring everyone online simultaneously. For instance, Sharaf could develop the AI service integration in one time zone while Scarlet worked on admin features in another, with changes merged asynchronously via pull requests.

3.3 Responsive Problem-Solving and Communication

Throughout both sprints, the team was highly responsive when issues arose. When we discovered that CurrentsAPI did not provide full article body content (a critical gap for our AI summarization pipeline), the team quickly pivoted to evaluating alternative APIs and settled on the Guardian API within the same sprint cycle. Similarly, when the initial plan to use a free LLM proved infeasible, the team pivoted to Google Gemini on the advisors' recommendation, keeping costs under control while unblocking the entire AI feature set.

4. What Didn't Work / Challenges

4.1 Delayed LLM Strategy Definition

The team spent most of Sprint 1 without a concrete plan for AI integration—the core differentiator of the platform. We explored free LLMs, considered Cohere, debated student Gemini credits, and discussed OpenAI budgets without converging on a decision. This indecision meant Sprint 1 ended with zero AI functionality, pushing the entire AI workload into Sprint 2. Root cause: the team treated the LLM decision as a research question rather than a time-boxed spike with a deadline. Our advisors had to explicitly recommend a budgeted OpenAI approach before the team committed to a direction.

4.2 Meeting Coordination Across Time Zones

In Sprint 2, with members spread across three time zones, coordinating synchronous meetings became a recurring friction point. Weekly standups continued, but scheduling additional working sessions or ad-hoc reviews was often difficult. Mid-sprint, alignment on the overall project direction drifted—individual tasks progressed, but the team lost a shared sense of priority ordering. Root cause: we did not establish fixed, non-negotiable meeting times at the start of Sprint 2. Instead, we tried to find times week-by-week, which was unpredictable and led to missed sync opportunities.

4.3 Scope Overcommitment in Sprint 2

The Sprint 2 readiness document planned for 16 stories totaling over 190 story points, including carry-overs, new AI features, feedback loops, notifications, and user uploads. This was significantly more than the team could realistically deliver in a single sprint. As a result, Epic 7 (Notifications & Alerts) was dropped entirely, Epic 6 (Feedback & Learning Loop) was only partially completed, and per-topic expertise levels were deferred. Root cause: the team did not use Sprint 1 velocity as a planning constraint. The desire to demonstrate a full feature set for the final submission led to optimistic commitment rather than disciplined scoping.

5. Lessons Learned & Changes We Would Make

If we could restart this project today, we would make the following changes:

5.1 Time-Box Technical Spikes Early

We would allocate the first two weeks of Sprint 1 to a formal, time-boxed spike for the AI/LLM integration. Rather than leaving the decision open-ended, we would prototype two or three candidate approaches (e.g., OpenAI, Gemini, Hugging Face inference) against a fixed budget, evaluate latency and output quality on sample articles, and commit to a stack before the end of the spike. This would have given us at least basic AI functionality by Sprint 1's end and spread the integration risk across both sprints.

5.2 Plan Sprints to Velocity, Not Ambition

We would track velocity from Sprint 1 (total completed story points) and use it as a hard cap for Sprint 2 planning. Stories beyond that cap would remain in the backlog, explicitly deprioritized. This would have prevented the overcommitment that led to dropping notifications and leaving the feedback loop incomplete. Honest velocity-based planning produces a smaller but fully finished increment, which is more valuable than a larger set of partial features.

5.3 Fix Distributed Meeting Cadence from Day One

Knowing that two members would be on exchange, we would establish fixed, non-negotiable meeting times at the very start of Sprint 2 (or even during the break between sprints), accounting for all time zones. We would also designate a rotating “meeting scribe” to post written summaries in Discord within 30 minutes of each meeting, ensuring that anyone who missed a session could still stay aligned.

5.4 Invest in Automated Testing Earlier

Our system testing was primarily manual, with a pytest suite planned but never made reliable due to time constraints. If we were starting over, we would set up a basic integration test suite (e.g., API endpoint smoke tests) in Sprint 1 and expand it incrementally. This would have caught regressions faster and given us more confidence during the rapid integration work of Sprint 2.

6. Individual Reflections

Vivek Jariwala

As Product Owner, this project gave me a much deeper appreciation for what it takes to translate high-level stakeholder feedback into concrete, actionable backlog items that a development team can pick up and deliver. Early on, I found myself writing user stories that were either too vague or too prescriptive, and learning to strike the right balance was a process that took most of Sprint 1 to get comfortable with. Technically, I grew the most in full-stack integration work. I spent significant time connecting the React frontend to our Express API with JWT-based authentication, debugging CORS issues, and working with Google Gemini's API to implement the article complexity switching feature. Before this project, I had only worked with APIs in isolated lab exercises, so building a real pipeline from external data source to database to frontend was a meaningful step up. On the professional side, the biggest growth area for me was learning to manage scope under pressure. There were features I was personally excited about, like per-topic expertise levels, that I had to consciously deprioritize because the team did not have the capacity to deliver them well. Saying "not this sprint" to ideas I cared about was uncomfortable, but it was the right call, and it is a discipline I know I will rely on in my career going forward.

Kaura Sankaran

One technical skill I learned was about automated email relays and managing the underlying data tables that feed into them. A big thing I did towards the end of the project was reintegrating the verification email and being able to set up alerts for the users to get notifications every Wednesday about their selected topics. I had to create another email and learn how to manage the server so that we wouldn't exceed our API limits with Mailjet. Additionally, I had to learn how to use a couple of different packages as wrappers that would automate the emails at set times, as long as the server was running. I'd done something similar at work, but using their framework; learning how to do it from scratch was a valuable experience and something I could see myself setting up later to run other scripts as opposed to just sending emails.

A big soft skill I learned was the importance of communication and having meetings. As the Scrum Master, I was responsible for a lot of the planning and timing of when things were going to get done, as well as when we were going to meet and check-in on internal deadlines. While the planning was fairly easy to lay out, I found in the second semester with two members of our

team on exchange with 6+ hour time changes, it was quite difficult to plan our communications, not only within our team, but with our advisors as well. When our semi-weekly meetings got pushed to the wayside due to members changing schedules, tasks started to fall through the cracks and the end goal seemed more and more daunting. We were able to chat more towards the end, and thus get our project to a serviceable level, but I really saw the difference in our team cohesion as well as output when we were able to communicate more frequently versus pushing meetings off.

Sharaf Syed

This project was my deepest exposure to integrating AI and ML services into a production-style web application, and it stretched my technical skills in ways I did not expect. I led the integration of Google Gemini for article summaries and explanations, built the embedding generation pipeline using gemini-embedding-001 with pgvector for storage, and worked with Hugging Face models for readability scoring. One of the biggest technical lessons was learning to manage API costs and latency in a real system. Early on, our Gemini calls were slow and expensive because the prompts were too long and unstructured. I spent a lot of time iterating on prompt design, reducing token usage while maintaining output quality, and that process taught me that working with LLMs is as much about engineering discipline as it is about understanding the models themselves. I also handled the system testing phase, executing all 12 test cases and documenting results. Writing structured test cases and being rigorous about pre-conditions and expected results was a good exercise in thinking systematically about quality. Professionally, the biggest lesson was about proactive communication. When the AI service integration hit unexpected latency issues midway through Sprint 2, my first instinct was to try to fix it quietly before the next standup. Instead, I raised it early, and the team was able to adjust plans and offer help. That experience convinced me that surfacing problems early, even when they feel embarrassing, is always better than letting them grow in silence.

Scarlet Wu

During this project, I gained hands-on experience setting up and working with a distributed database using Yugabyte. Since this was my first time using the platform, the biggest challenge wasn't the complexity of the system itself, but rather figuring out how to navigate it like where to find cluster management, configuration settings, and how to properly set everything up through the online console. A lot of it came down to getting comfortable with an unfamiliar cloud-based platform and learning how to independently find the right resources and documentation.

Through this process, I became much more confident in picking up new tools and environments, which I know will be really important in future projects and in industry, where you're often expected to learn systems quickly on your own.

In terms of professional skills, I definitely strengthened my communication and coordination in a distributed team environment. A major challenge was collaborating across different countries and time zones, especially while I was based in Singapore. This meant being more intentional about communication clearly documenting progress, proactively updating teammates, and being flexible with meeting times. I also learned the value of quickly jumping on short calls to resolve blockers, instead of relying only on asynchronous communication.

7. Action Items for Future Projects

1. **Conduct time-boxed technical spikes for high-risk integrations:** Allocate a fixed 1–2 week window at the start of any project involving unfamiliar technology (e.g., LLMs, third-party APIs) to prototype, evaluate, and commit to an approach. Do not leave critical technology decisions open past the spike deadline.
2. **Use measured velocity as the sprint planning ceiling:** After the first sprint, calculate the team's actual throughput in story points and use it as a hard constraint for subsequent sprint commitments. Resist the temptation to plan to ambition rather than capacity.
3. **Establish meeting norms for distributed work on day one:** When working across time zones, fix meeting times before the sprint starts, designate a rotating scribe, and post written summaries within 30 minutes. Treat these norms as non-negotiable.
4. **Integrate automated testing from Sprint 1:** Set up at least API-level smoke tests in the first sprint and expand coverage incrementally. Manual-only testing does not scale and introduces risk during rapid integration phases.
5. **Schedule mid-sprint alignment check-ins:** Beyond standups, hold a brief mid-sprint review (30 minutes) to confirm that individual work still aligns with the sprint goal and overall project direction. This prevents drift when team members are working independently.